

Process Scheduling

Module 4

Operating Systems

Chapter Outline

- Scheduler
 - Scheduling algorithm
 - Objectives of Scheduling
 - Criteria for scheduling
 - CPU-bound vs I/O-bound processes
- Types of Scheduling
- Process scheduling queues
 - FCFS
 - SJF
 - Priority
 - Round Robin
 - Multilevel feedback queues scheduling
- BSD Unix scheduling
- Multiple processor scheduling

Scheduling

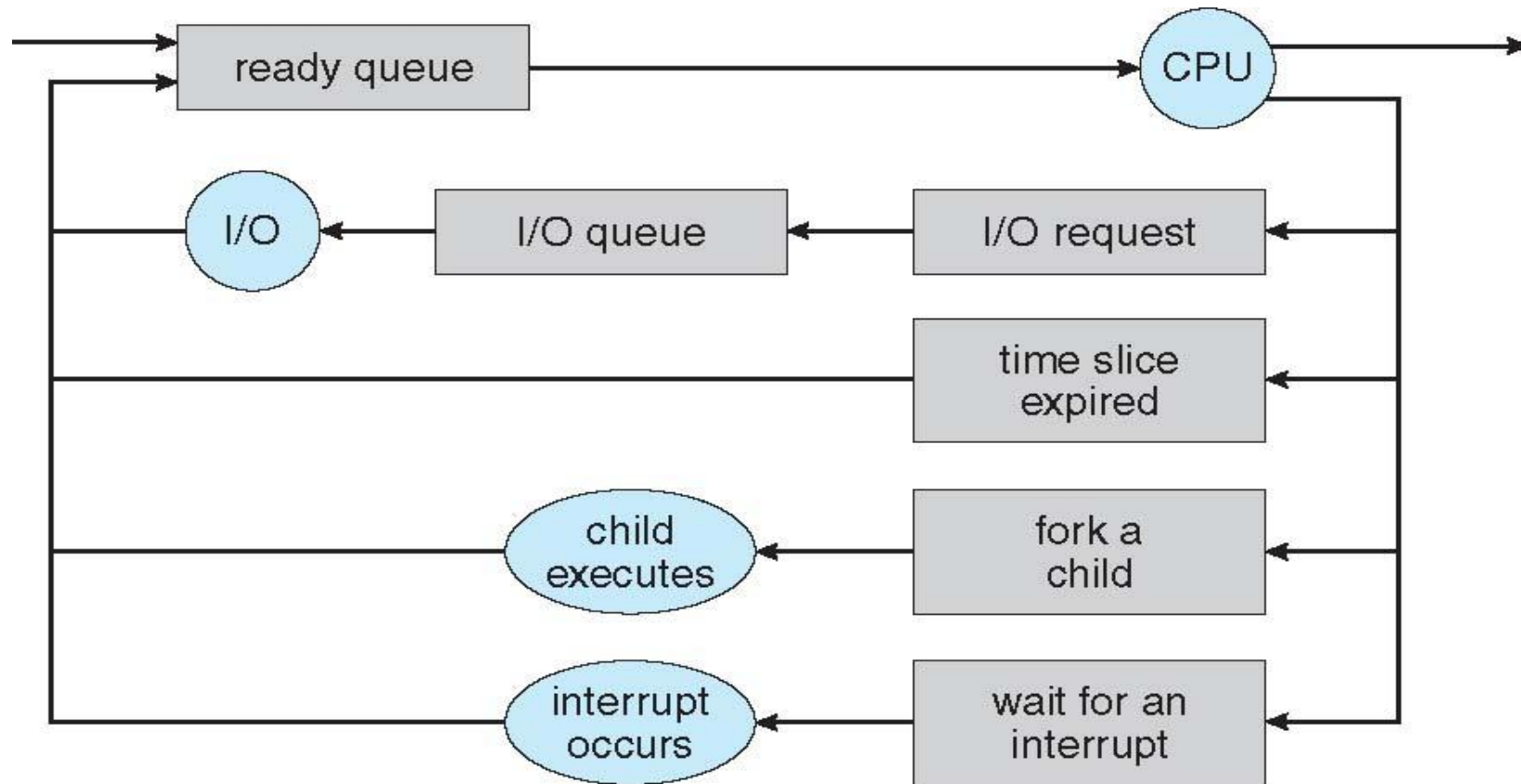
- Scheduler

- The scheduler is part of the operating system, concerned with which process to run next if multiple run-able processes are in ready queue
- Scheduler is concerned with deciding on policy, not providing a mechanism
- It is important because it can have a big effect on resource utilization and overall performance of the system

- Scheduling Algorithm

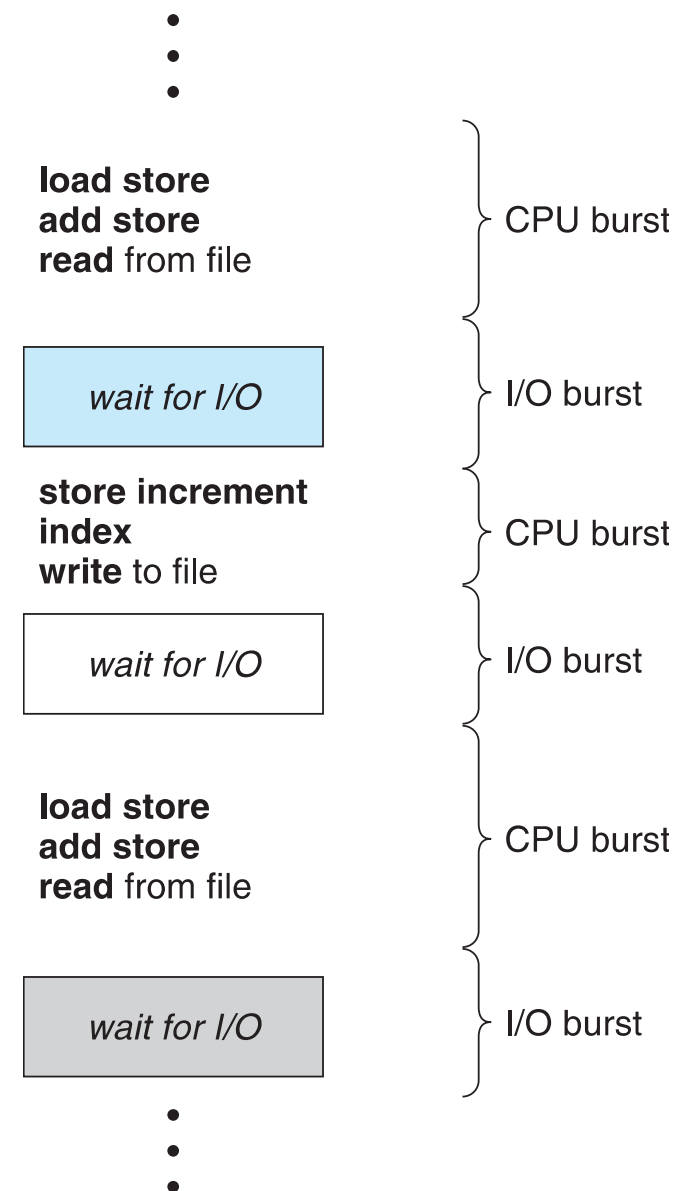
- In a multiprogramming computer several processes will compete for use of the processor
- At any time only one process will be running while other will wait for process to be free
- Scheduling algorithm is logic that determines the order in which processes should run when multiple processes are there
- The aim of process scheduling is to achieve objectives such as maximum CPU utilization and to improve its performance

Process Scheduling

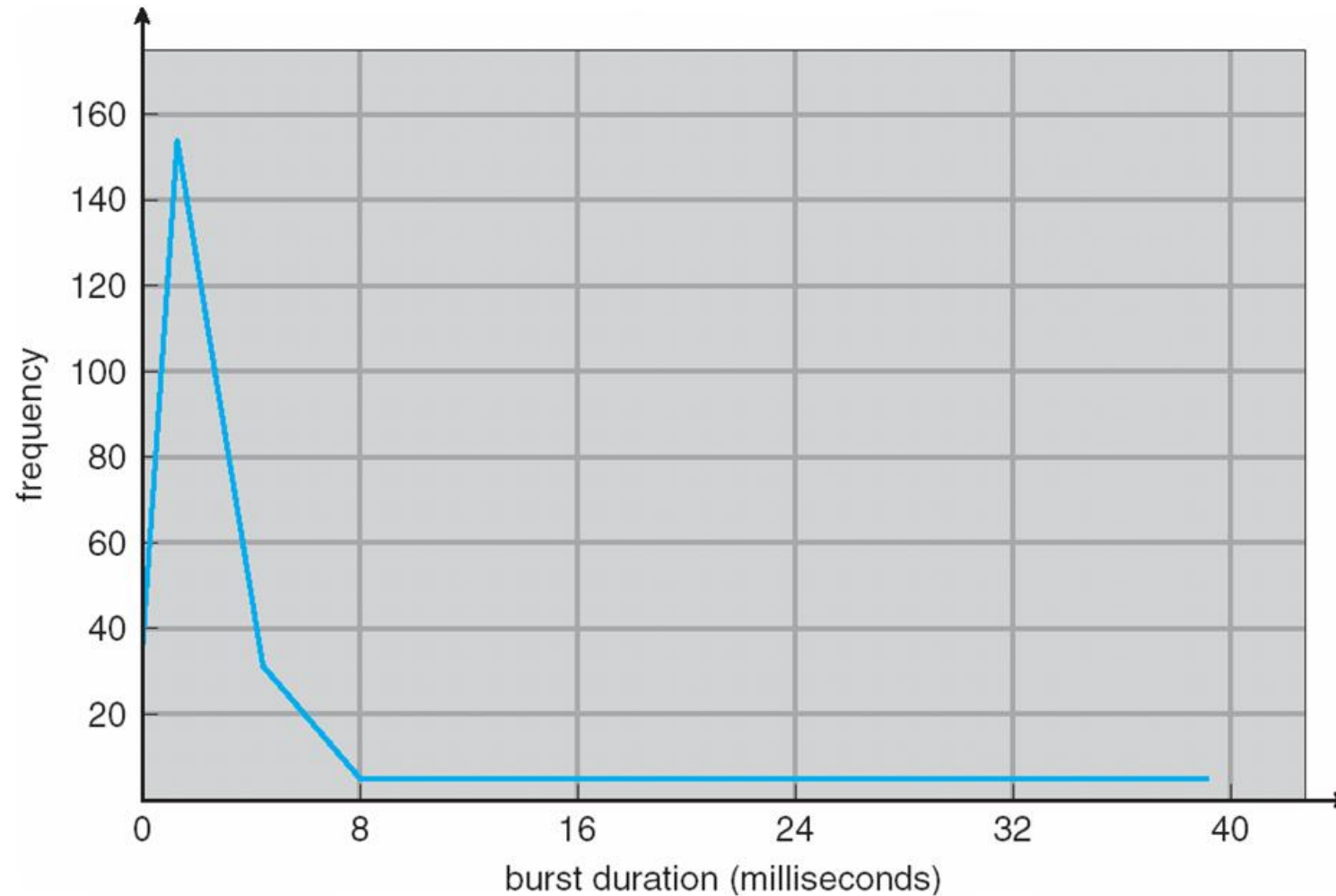


Process Scheduling

- Maximum CPU utilization obtained with multiprogramming
- CPU–I/O Burst Cycle – Process execution consists of a cycle of CPU execution and I/O wait
- CPU burst followed by I/O burst
- CPU burst distribution is of main concern



Histogram of CPU-burst Times



CPU Scheduler

- Short-term scheduler selects from among the processes in ready queue, and allocates the CPU to one of them
 - Queue may be ordered in various ways
- CPU scheduling decisions may take place when a process:
 - Switches from running to waiting state
 - Switches from running to ready state
 - Switches from waiting to ready
 - Terminates
- Scheduling under 1 and 4 is non-preemptive
- All other scheduling is preemptive
 - Consider access to shared data
 - Consider preemption while in kernel mode
 - Consider interrupts occurring during crucial OS activities

Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:
 - switching context
 - switching to user mode
 - jumping to the proper location in the user program to restart that program
- Dispatch latency
 - time it takes for the dispatcher to stop one process and start another running

Scheduling Performance

- CPU utilization
 - keep the CPU as busy as possible
- Throughput
 - # of processes that complete their execution per time unit
- Turnaround time
 - amount of time to execute a particular process
- Waiting time
 - amount of time a process has been waiting in the ready queue
- Response time
 - amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)

Scheduling Algorithm Objectives

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

Criteria for Scheduling

- Priority
 - Priority assigned to that process or job
- Class
 - Class of job, i.e. batch or on-line or real-time
- Resource requirements
 - E.g. expected run time, memory required, etc
- I/O or CPU bound
 - i.e. whether job is I/O bound or CPU bound
- Resources used to date
 - i.e. the amount of processor time already consumed
- Waiting time to date
 - i.e. the amount of time spent waiting for service so far

Scheduling Philosophies

- Preemptive
 - The system may preempt the CPU from a process at any time;
 - Prevents any one process from using the CPU for too long
 - Can lead to race condition (require synchronization techniques)
- Non-preemptive
 - Each process voluntarily gives up the CPU
 - It is;
 - Simple
 - Easy to implement
 - Not suitable for multi-user systems
 - If the running process become blocked next process can be scheduled

Process Scheduling Queues

- Job queue
 - Set of all the processes in the system
 - It also includes the processes from the queues given below
- Ready queue
 - Set of all the processes residing in main memory (ready or waiting)
- Device queue
 - Set of processes waiting for an I/O devices

Levels of Scheduling

- Scheduling can be exercised at three levels
- High-level scheduling (Long-term scheduler)
 - Allow limited number of processes in the ready queue to compete
 - It disallows processes beyond a certain limit for batch processes first and in the end also the interactive processes
 - Invoke less frequently
 - May be slow
- Medium-level scheduling (Medium-term scheduler)
 - It is concerned with decision to temporarily remove a process from the system (to reduce system load)
 - To reintroduce a process or swapping in
 - What if there is room in memory and both a new process as well as swapped out process want to be loaded
 - In this case, medium level scheduler has to work in close conjunction with high level scheduler

Levels of Scheduling

- Low level-scheduling (Short-term scheduler)
 - Handles the decisions of which process is to be assigned to the processor (dispatcher)
 - Invoke very frequently
 - Must execute fast
 - Its scheduling could be preemptive or non-preemptive
 - Dispatch latency should be minimum
 - Dispatch latency is the time it takes for the dispatcher to stop one process and start another running
 - Example:
 - Assume a process runs for 90ms and scheduler run for 10ms
 - Overheads = $10/(90+10) = 10\%$
- Any OS may use one or all of these levels, depending upon the desired
- These levels have to interact amongst themselves quite closely to ensure that the computing resources are managed optimally
- The exact algorithms for these and interaction between them are quite complex

Schedulers in Operating System



Scheduler	Key Decision	Runs On	Frequency
Long-Term	Which jobs to admit	Disk → Main Memory	Low
Medium-Term	Which jobs to suspend/resume	Main Memory ↔ Disk	Medium
Short-Term	Which process gets CPU	Ready Queue → CPU	High

Scheduling Algorithms

First Come First Serve Scheduling

- The FCFS simply assign the processor to the process which is first in ready queue
- When the processor is free, the next process at the head of the ready queue will be selected

- Consider the following set of three processes that arrive at time '0'

Process ID	Priority	Burst Time
P1	2	30 ms
P2	3	6 ms
P3	1	4 ms



- The waiting time for each process is P1=0, P2=30, P3=36
- The average waiting time is $(0+30+36)/3 = 22\text{ms}$
- The average waiting time of FCFS may vary substantially if the process's CPU burst time vary greatly

First Come First Serve Scheduling

- Salient points of FCFS
 - It is the simplest scheduling algorithm
 - Also known as the first in first out (FIFO)
 - FCFS is non-preemptive
 - It performs much better for long processes than short processes
 - It tends to favor CPU-bound processes over I/O bound
 - It is not suitable for time sharing system
 - FCFS is rarely used on its own but is effective if combined with other scheme

Practice Example (FCFS)

Process	Arrival Time (ms)	Burst Time (ms)
P1	0	5
P2	2	3
P3	4	8

1. Sort Processes by Arrival Time
2. Design Gantt Chart
3. Calculate Waiting Time & Turnaround Time
4. Calculate Averages (Average waiting & Turnaround Time)

Solution

Step 1: Sort Processes by Arrival Time

- Since FCFS executes the process that **arrives first**, we arrange them by **arrival order**:

P1 → P2 → P3

Step 2: Gantt Chart

Let's schedule them in order:

P1	P2	P3
0 5	8	16

Step 3: Calculate Waiting Time & Turnaround Time

Process	Arrival Time	Burst Time	Start Time	Finish Time	Waiting Time	Turnaround Time
P1	0	5	0	5	0	5
P2	2	3	5	8	$5 - 2 = 3$	$8 - 2 = 6$
P3	4	8	8	16	$8 - 4 = 4$	$16 - 4 = 12$

Step 4: Calculate Averages

- **Average Waiting Time** = $(0 + 3 + 4) / 3 = 2.33$ ms
- **Average Turnaround Time** = $(5 + 6 + 12) / 3 = 7.67$ ms

Shortest Job First Scheduling

- This algorithm associates with shortest job first
- When the CPU is available, it is assigned to the process that has smallest CPU burst time
- If two processes have same CPU-burst then FCFS scheduling is used along with this to break the tie
- Moving a short process before a long one
 - Decrease the waiting time of short process
 - Increase the waiting time of long process
 - Consequently, the average time decreases
- This will result in a smaller number of PCB's in the ready or blocked queues
- The search time will be smaller, thus improving the response time
- The number of satisfied user will increase
- A long job in the queue may delayed indefinitely by a succession of smaller jobs arriving in ready queue
- This can be avoided by setting higher external priority to those important jobs
- The OS at any time can calculates a resultant priority based on both
- Also known as shortest job next

Shortest Job First Scheduling

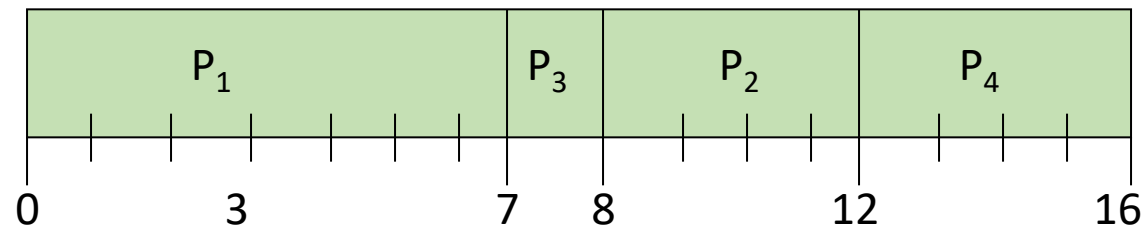
- Salient Points of SJF
 - SJF may be preemptive or non-preemptive
 - It will preempt the currently executing process if its remaining time is greater than that of a newly arrived process
 - This scheme is known as shortest remaining time first
 - SJF is optimal in sense that it yields the smallest average waiting time
 - In general, it is difficult to predict the CPU time requirement for a process
 - SJF is optimal for batch jobs for which the run times are known in advance
 - SJF reduces average waiting time over FCFS

Shortest Job First Scheduling

Non-Preemptive SJF Scheduling

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (non-preemptive)



- Average waiting time = $(0 + 6 + 3 + 7)/4 = 4$

Completion, Turnaround and Waiting times

Turnaround = Completion – Arrival

Waiting = Turnaround – Burst

Process	Arrival	Burst	Completion C	Turnaround (C-A)	Waiting (TAT – Burst)
P ₁	0	7	7	7 – 0 = 7	7 – 7 = 0
P ₂	2	4	12	12 – 2 = 10	10 – 4 = 6
P ₃	4	1	8	8 – 4 = 4	4 – 1 = 3
P ₄	5	4	16	16 – 5 = 11	11 – 4 = 7

Totals / averages:

- Average **turnaround time** = $(7 + 10 + 4 + 11) / 4 = \mathbf{8}$ units
- Average **waiting time** = $(0 + 6 + 3 + 7) / 4 = \mathbf{4}$ units

Final answer

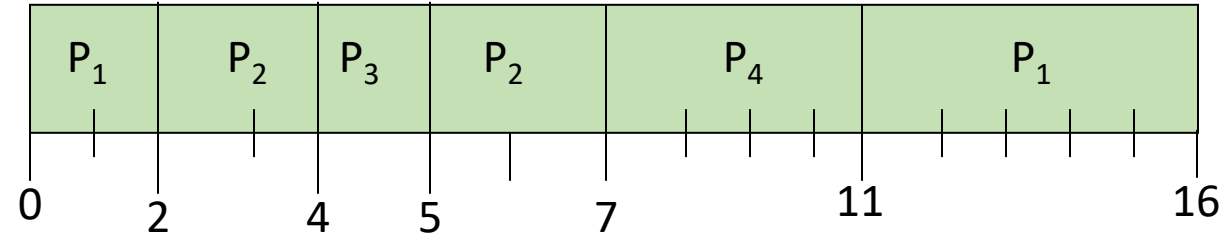
- Average waiting time = **4 units**
- Average turnaround time = **8 units**

Shortest Job First Scheduling

Preemptive SJF Scheduling

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (preemptive)



Completion, Turnaround and Waiting times

Compute completion time (C), turnaround = C – arrival, waiting = turnaround – burst.

Process	Arrival	Burst	Completion C	Turnaround (C-A)	Waiting (TAT-Burst)
P ₁	0	7	16	16	16 – 7 = 9
P ₂	2	4	7	7 – 2 = 5	5 – 4 = 1
P ₃	4	1	5	5 – 4 = 1	1 – 1 = 0
P ₄	5	4	11	11 – 5 = 6	6 – 4 = 2

Totals / averages:

- Average **turnaround time** = $(16 + 5 + 1 + 6) / 4 = 28 / 4 = 7$
- Average **waiting time** = $(9 + 1 + 0 + 2) / 4 = 12 / 4 = 3$

Average waiting time = **3** units. Average turnaround time = **7** units.

Round Robin Scheduling

- The Round-Robin (RR) scheduling is designed especially for time sharing systems
- In this algorithm there is small unit of time known as time quantum
- A time quantum is generally up to 10-100ms
- CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 quantum

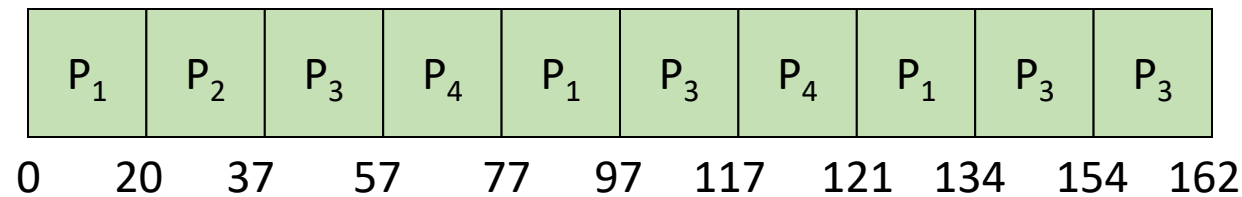
Round Robin Scheduling

- For RR scheduling we keep the ready queue as a FIFO queue of processes
- The CPU picks up the first process from the ready queue, set a timer to interrupt after 1 quantum
- If the process burst time is less than 1 quantum
 - then the process itself release the CPU voluntarily & proceed to next process
- If the process burst time is greater than 1 quantum
 - then that process is sent back to the ready queue and CPU scheduler selects the next process in the ready queue for scheduling

Round Robin Scheduling

<u>Process</u>	<u>Burst Time</u>
P ₁	53
P ₂	17
P ₃	68
P ₄	24

- Time quantum = 20 ms
- The Gantt chart is:



- Typically, higher average turnaround than SJF, but better response

Priority Scheduling

- In priority scheduling a priority is associated with each process and the CPU is allocated to the process with the highest priority
- Equal priority processes are scheduled in FCFS order
- Priorities are generally some fixed range of numbers, such as 0, 1, 5, 10
- There is no general agreement on whether '0' is highest or lower priority
- Priority can be defined on the bases of following
 - Process time limit
 - Memory requirements
 - Number of open files
 - Ratio of average I/O burst to CPU burst

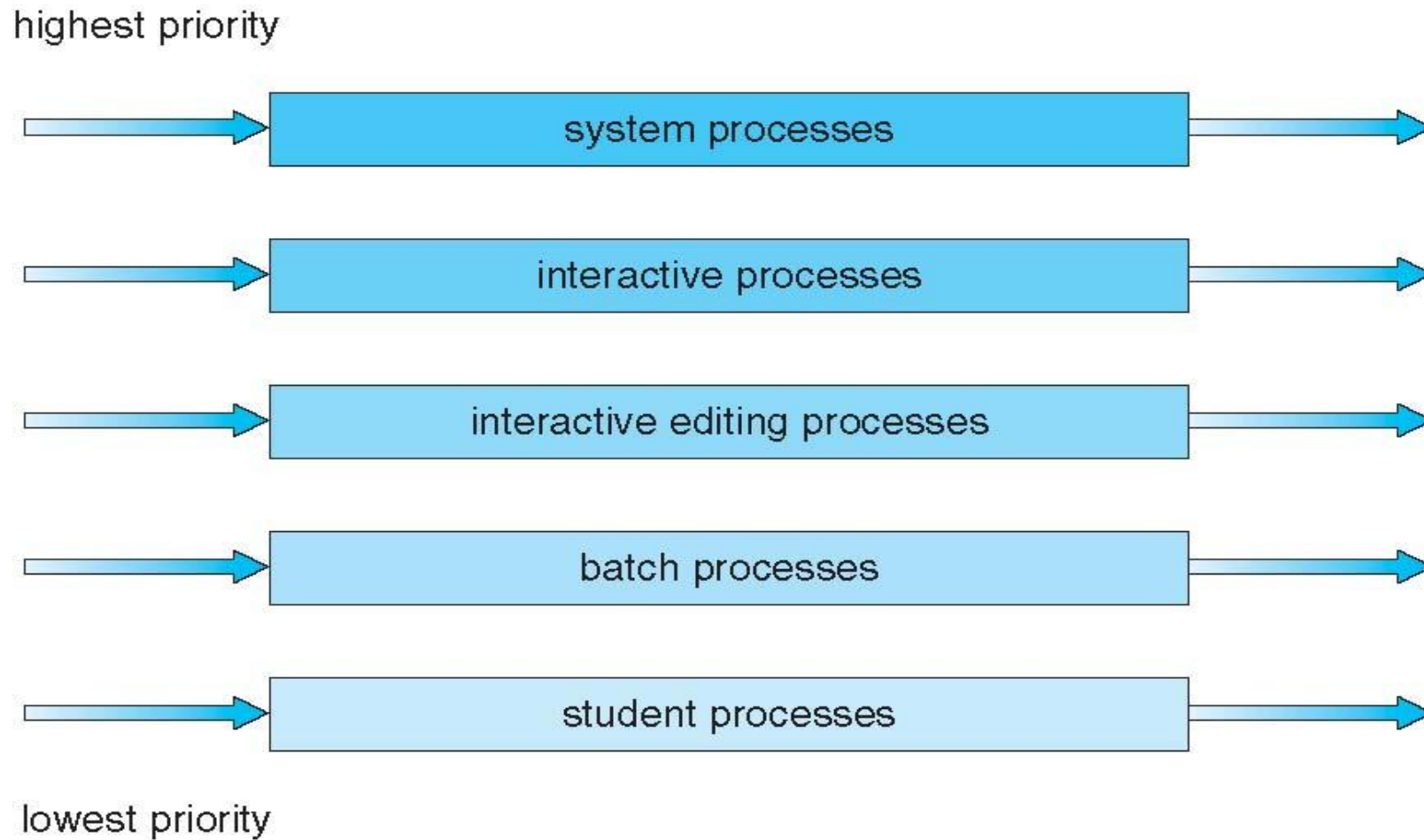
Priority Scheduling - Salient points

- Priority mechanism can be static or dynamic
 - Static priority mechanism is easy to implement and have relatively low overheads
 - Dynamic priority mechanism is more complex to implement and have greater overheads
- The priority scheduling may be preemptive or non-preemptive
 - The choice arise when a high priority process comes in the queue while a low priority process is executing
- A process that is ready to run but lacking the CPU can be considered blocked
- Solution to indefinite blocking is aging
- Aging is a technique of gradually increasing the priority of processes that waits for a long time
- OS allows only a limited priority classes
- In VAX / VMS priority range is 0 to 31
- In Unix priority range is -20 to 20

Multilevel Queue

- Ready queue is partitioned into separate queues:
 - foreground (interactive)
 - background (batch)
- Each queue has its own scheduling algorithm
 - foreground – RR
 - background – FCFS
- Scheduling must be done between the queues
 - Fixed priority scheduling; (i.e., serve all from foreground then from background)
 - Possibility of starvation
 - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR
 - 20% to background in FCFS

Multilevel Queue Scheduling



Multilevel Feedback Queue

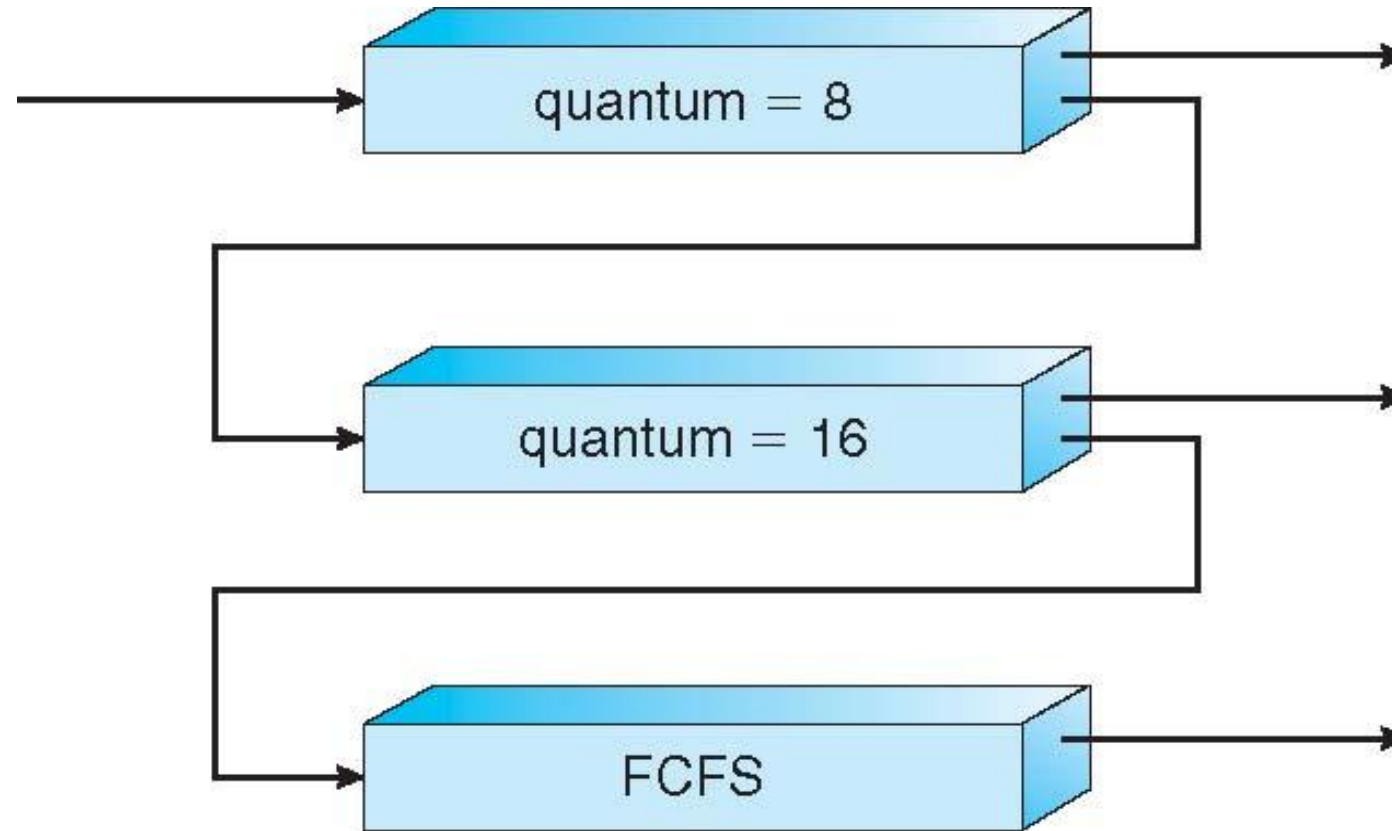
- A process can move between the various queues
 - aging can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters:
 - number of queues
 - scheduling algorithms for each queue
 - method used to determine when to upgrade a process
 - method used to determine when to demote a process
 - method used to determine which queue a process will enter when that process needs service

Multilevel Feedback Queue

Example

- Three queues:
 - Q_0 – time quantum 8 milliseconds
 - Q_1 – time quantum 16 milliseconds
 - Q_2 – FCFS
- Scheduling
 - A new job enters queue Q_0 which is served FCFS
 - When it gains CPU, job receives 8 milliseconds
 - If it does not finish in 8 milliseconds, job is moved to queue Q_1
 - At Q_1 job is again served FCFS and receives 16 additional milliseconds
 - If it still does not complete, it is preempted and moved to queue Q_2

Multilevel Feedback Queues



Thread Scheduling

Thread Scheduling

- Distinction between user-level and kernel-level threads
- When threads supported, threads scheduled, not processes
- **Process-contention scope (PCS)**
 - Thread library schedules user-level threads to run on LW
 - Scheduling competition is within the process
 - Typically done via priority set by programmer
 - Many-to-one and many-to-many models
- **System-contention scope (SCS)**
 - Kernel thread scheduled onto available CPU
 - Competition among all threads in system

Pthread Scheduling

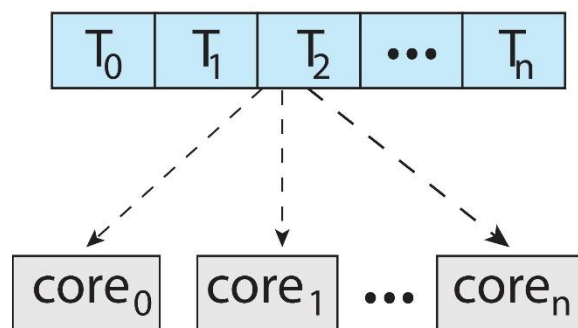
- API allows specifying either PCS or SCS during thread creation
 - PTHREAD_SCOPE_PROCESS schedules threads using PCS scheduling
 - PTHREAD_SCOPE_SYSTEM schedules threads using SCS scheduling
- Can be limited by OS
 - Linux and macOS only allow PTHREAD_SCOPE_SYSTEM

Multiple-Processor Scheduling

- CPU scheduling more complex when multiple CPUs are available
- Multiprocessor may be any one of the following architectures:
 - Multicore CPUs
 - Multithreaded cores
 - NUMA systems
 - Heterogeneous multiprocessing

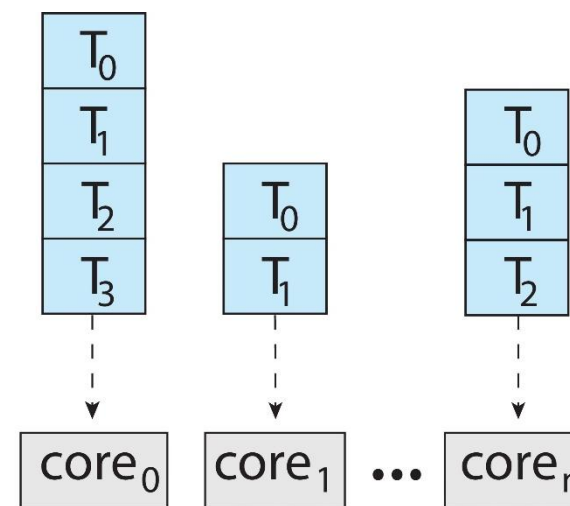
Multiple-Processor Scheduling

- Symmetric multiprocessing (SMP) is where each processor is self scheduling
- All threads may be in a common ready queue (a)
- Each processor may have its own private queue of threads (b)



common ready queue

(a)

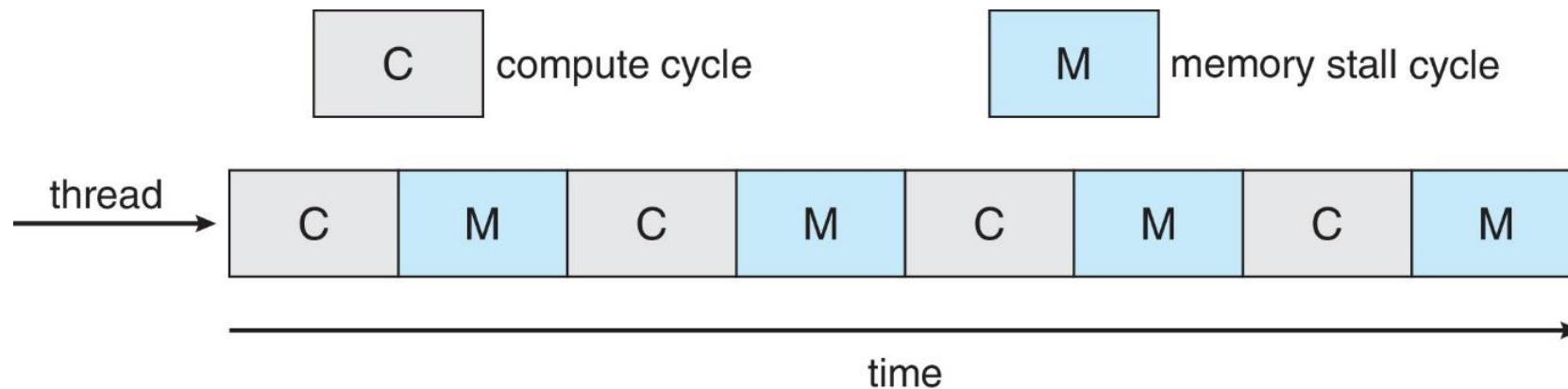


per-core run queues

(b)

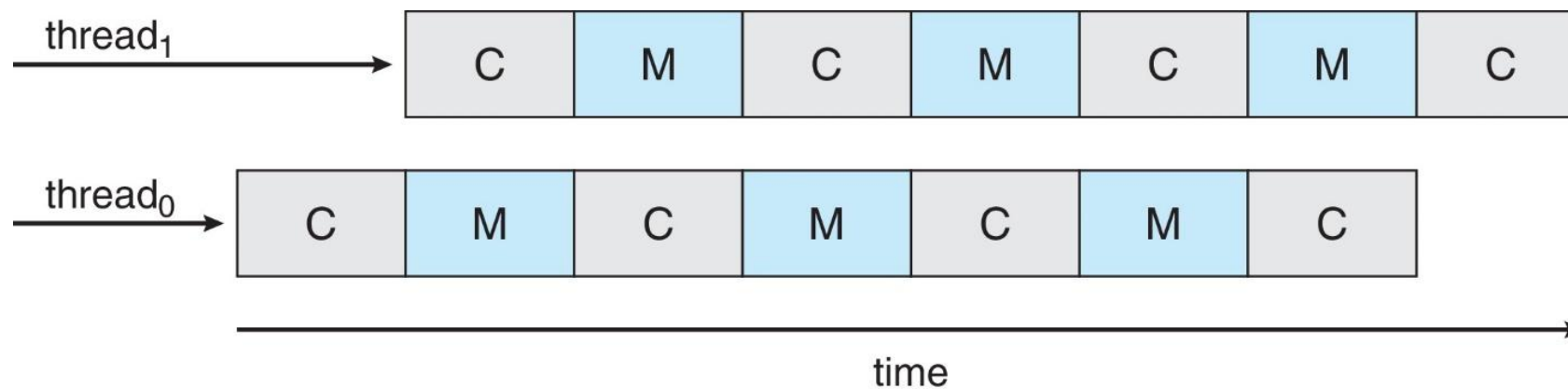
Multicore Processors

- Recent trend to place multiple processor cores on same physical chip
- Faster and consumes less power
- Multiple threads per core also growing
 - Takes advantage of memory stall to make progress on another thread while memory retrieve happens



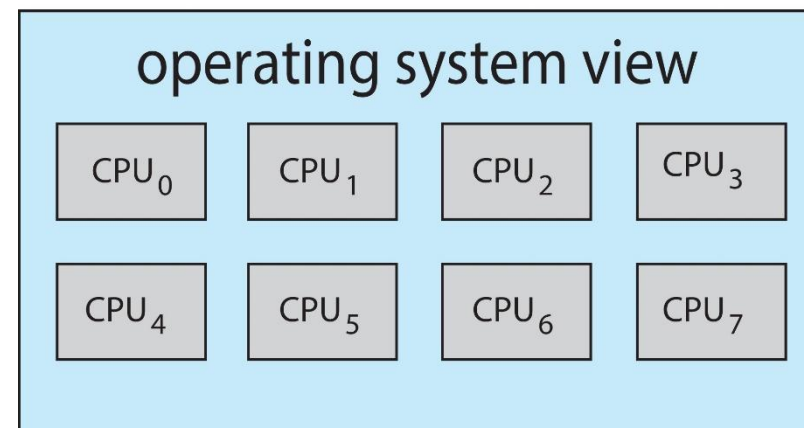
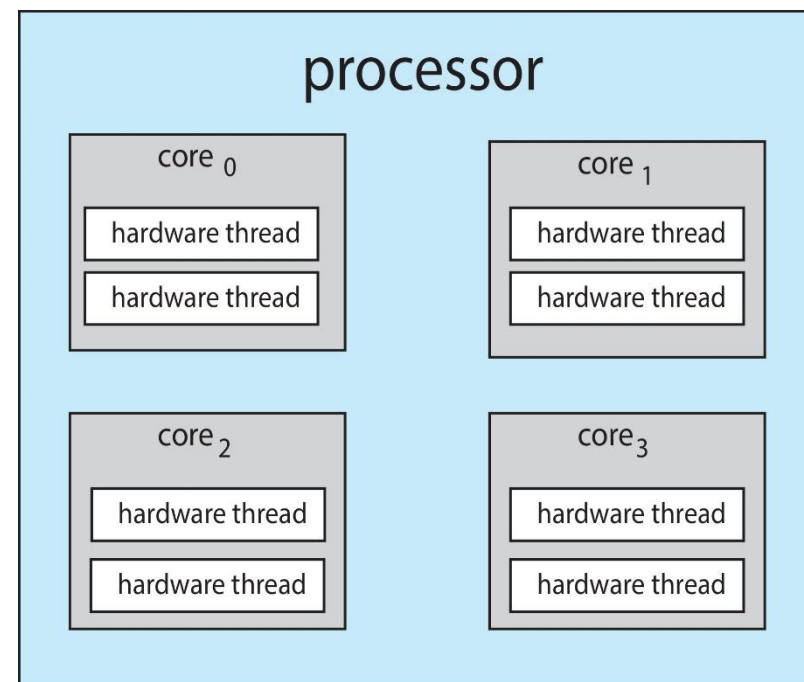
Multithreaded Multicore System

- Each core has > 1 hardware threads
- If one thread has a memory stall, switch to another thread



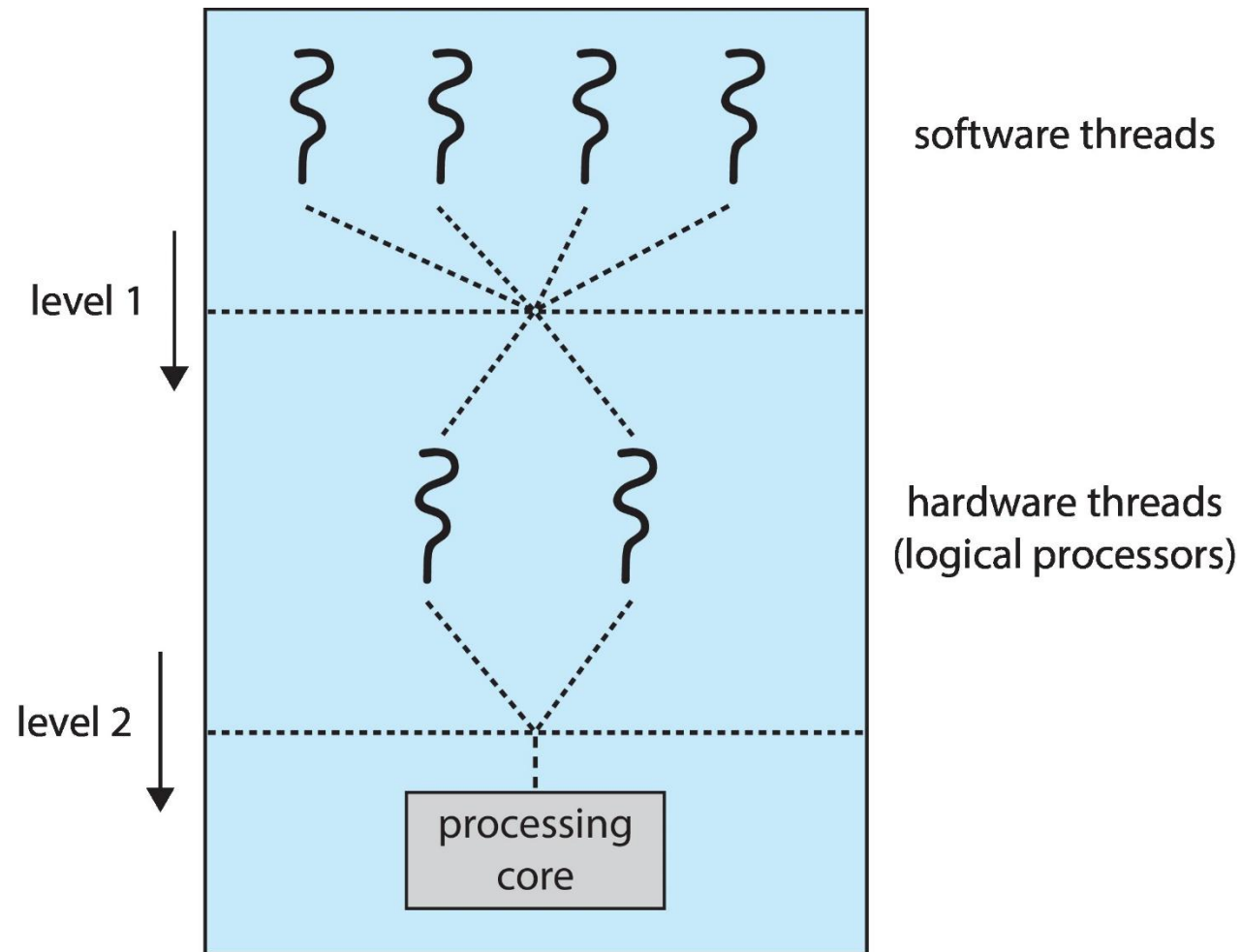
Multithreaded Multicore System

- Chip-multithreading (CMT) assigns each core multiple hardware threads
 - Intel refers to this as hyperthreading
- On a quad-core system with 2 hardware threads per core, the operating system sees 8 logical processors



Multithreaded Multicore System

- Two levels of scheduling:
 - The operating system deciding which software thread to run on a logical CPU
 - How each core decides which hardware thread to run on the physical core



Multiple-Processor Scheduling – Load Balancing

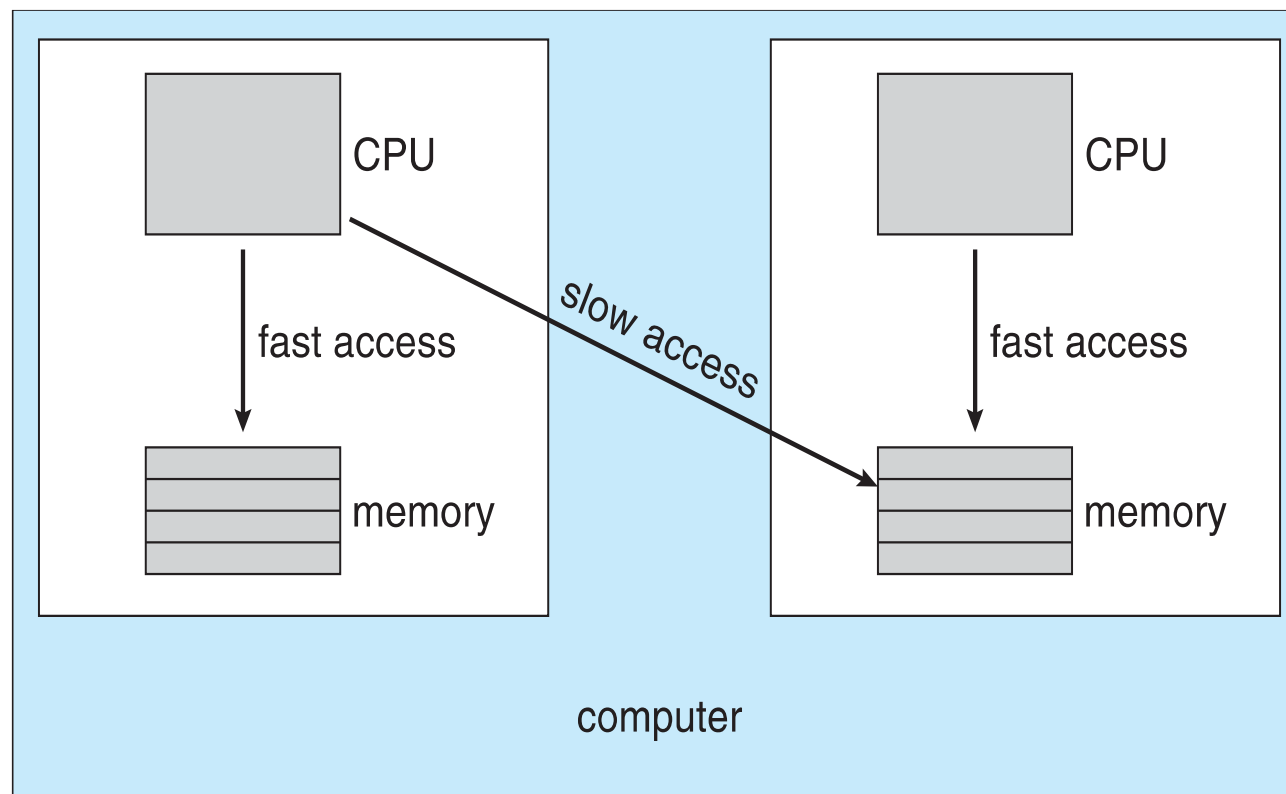
- If SMP, need to keep all CPUs loaded for efficiency
- Load balancing
 - attempts to keep workload evenly distributed
- Push migration
 - periodic task checks load on each processor, and if found pushes task from overloaded CPU to other CPUs
- Pull migration
 - idle processors pulls waiting task from busy processor

Multiple-Processor Scheduling – Processor Affinity

- When a thread has been running on one processor, the cache contents of that processor stores the memory accesses by that thread
- We refer to this as a thread having affinity for a processor (i.e., “processor affinity”)
- Load balancing may affect processor affinity as a thread may be moved from one processor to another to balance loads, yet that thread loses the contents of what it had in the cache of the processor it was moved off of
- Soft affinity
 - the operating system attempts to keep a thread running on the same processor, but no guarantees
- Hard affinity
 - allows a process to specify a set of processors it may run on

NUMA and CPU Scheduling

- If the operating system is **NUMA-aware**, it will assign memory close to the CPU the thread is running on



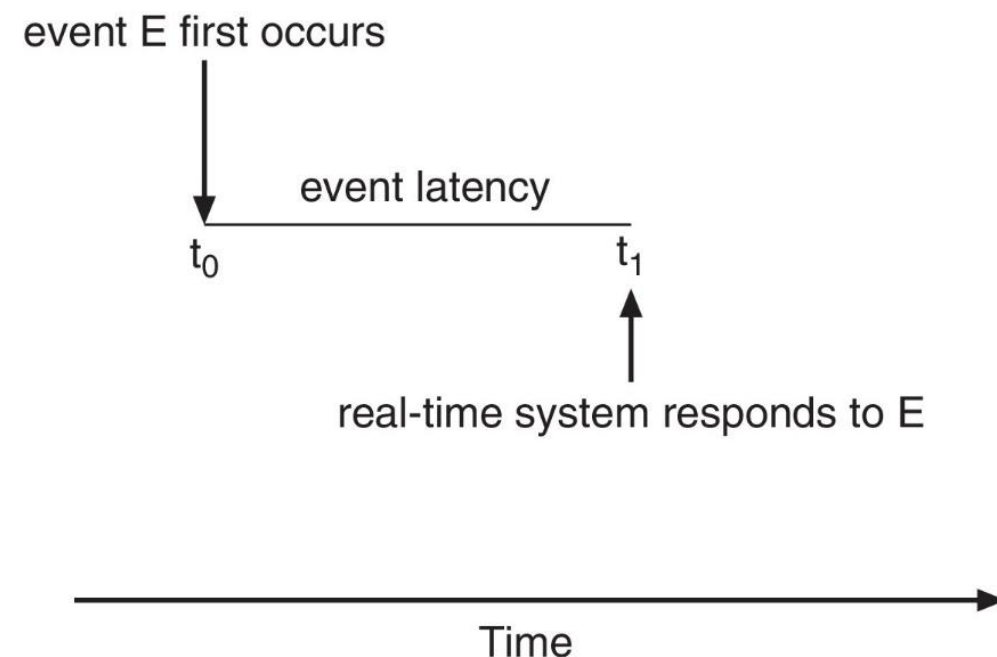
Real-Time Process Scheduling

Real-Time CPU Scheduling

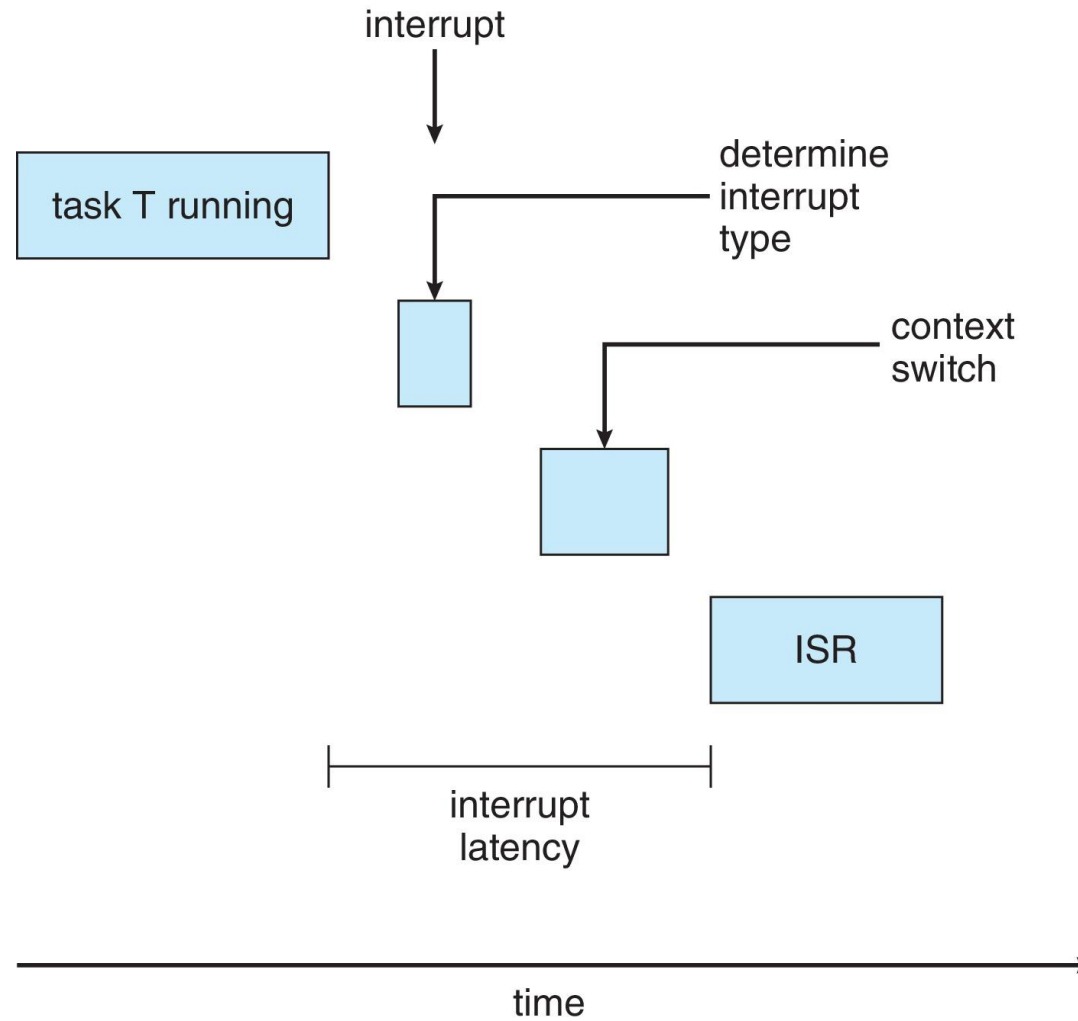
- Can present obvious challenges
- Soft real-time systems
 - Critical real-time tasks have the highest priority, but no guarantee as to when tasks will be scheduled
- Hard real-time systems
 - task must be serviced by its deadline

Real-Time CPU Scheduling

- Event latency – the amount of time that elapses from when an event occurs to when it is serviced
- Two types of latencies affect performance
 - Interrupt latency
 - time from arrival of interrupt to start of routine that services interrupt
 - Dispatch latency
 - time for schedule to take current process off CPU and switch to another

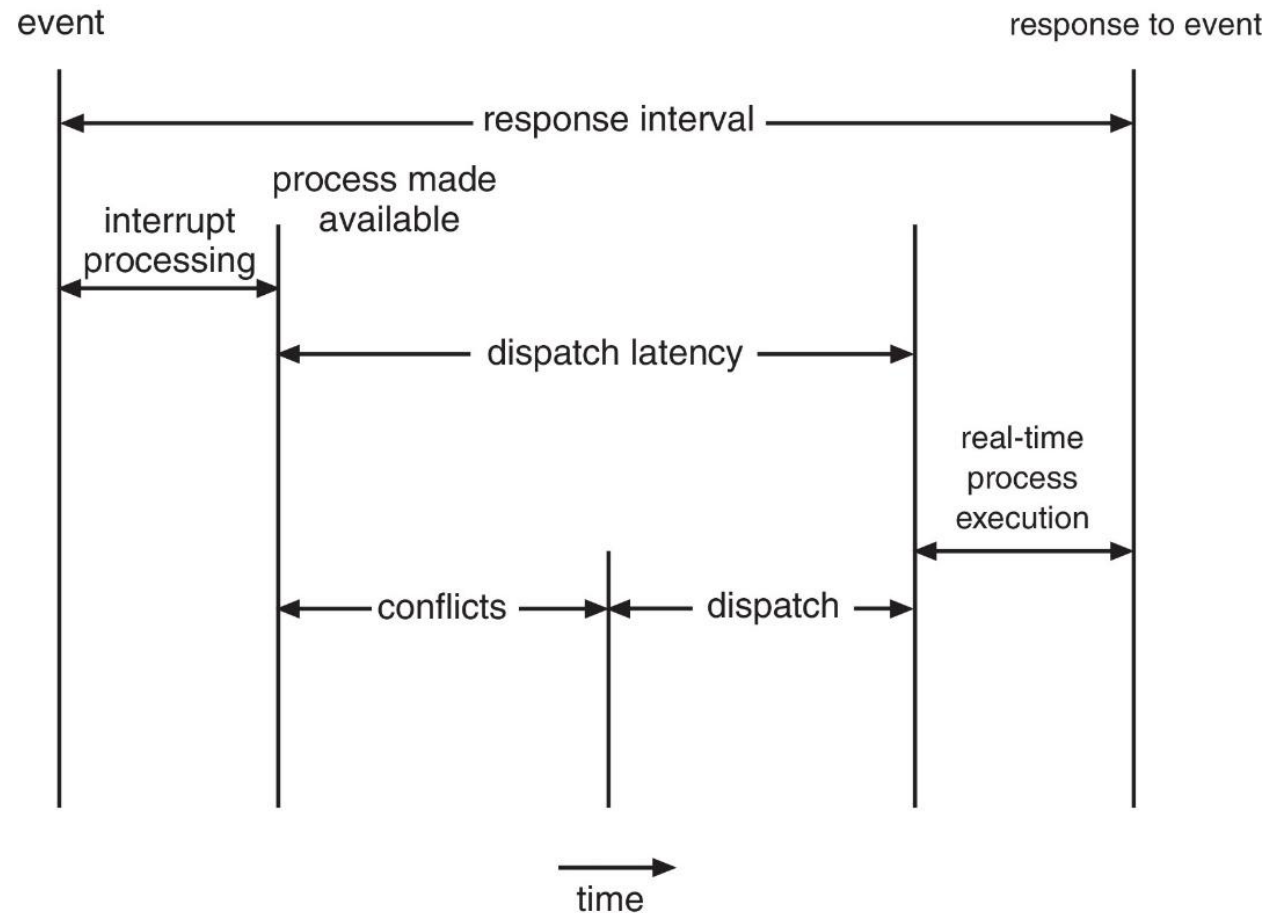


Interrupt Latency



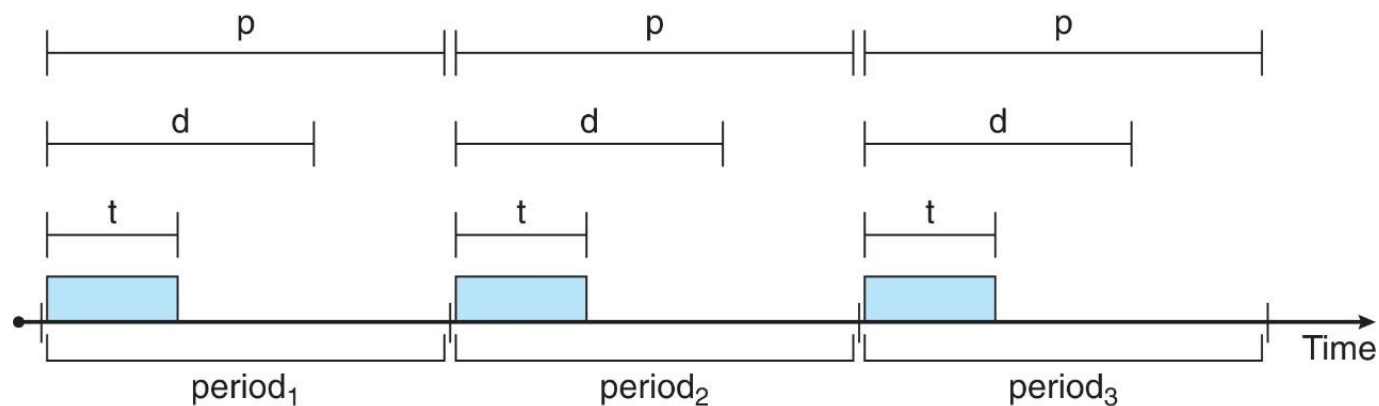
Dispatch Latency

- Conflict phase of dispatch latency
 - Preemption of any process running in kernel mode
 - Release by low-priority process of resources needed by high-priority processes



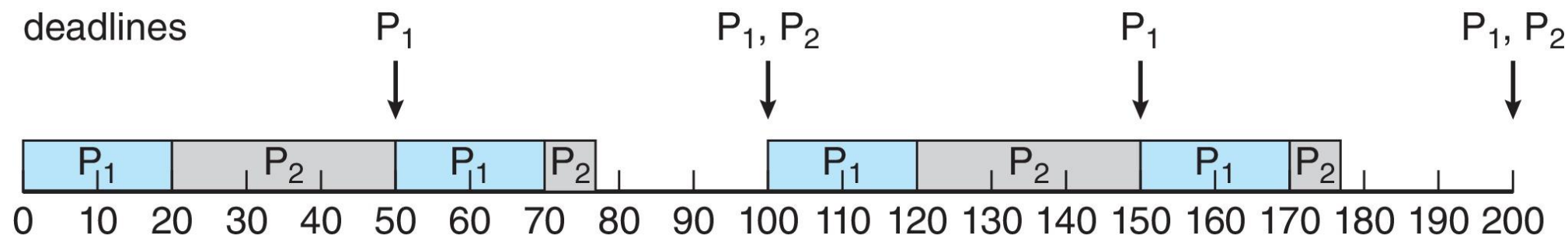
Priority-based Scheduling

- For real-time scheduling, scheduler must support preemptive, priority-based scheduling
 - But only guarantees soft real-time
- For hard real-time must also provide ability to meet deadlines
- Processes have new characteristics: periodic ones require CPU at constant intervals
 - Has processing time t , deadline d , period p
 - $0 \leq t \leq d \leq p$
 - Rate of periodic task is $1/p$



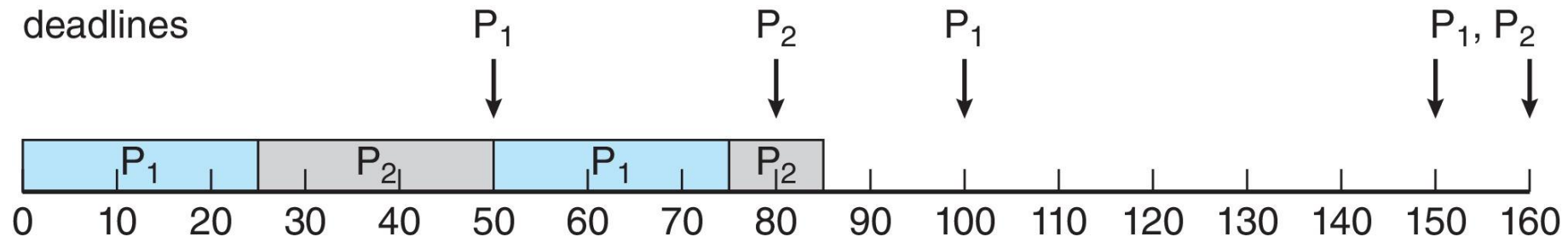
Rate Monotonic Scheduling

- A priority is assigned based on the inverse of its period
- Shorter periods $\rightarrow$ higher priority
- Longer periods $\rightarrow$ lower priority
- P1 is assigned a higher priority than P2



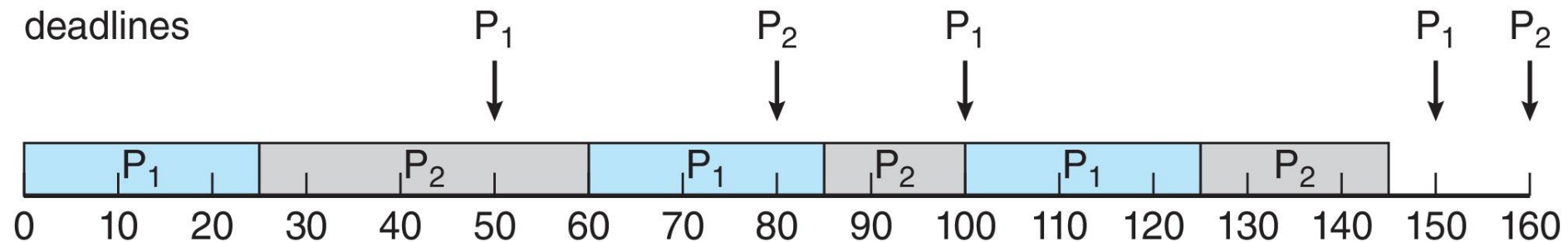
Missed Deadlines with Rate Monotonic Scheduling

- Process P2 misses finishing its deadline at time 80



Earliest Deadline First Scheduling (EDF)

- Priorities are assigned according to deadlines
 - The earlier the deadline, the higher the priority
 - The later the deadline, the lower the priority



Proportional Share Scheduling

- T shares are allocated among all processes in the system
- An application receives N shares where $N < T$
- This ensures each application will receive N / T of the total processor time

POSIX Real-Time Scheduling

- The POSIX.1b standard
- API provides functions for managing real-time threads
- Defines two scheduling classes for real-time threads
 1. SCHED_FIFO - threads are scheduled using a FCFS strategy with a FIFO queue
 - There is no time-slicing for threads of equal priority
 2. SCHED_RR - similar to SCHED_FIFO except time-slicing occurs for threads of equal priority
- Defines two functions for getting and setting scheduling policy:
 1. `pthread_attr_getsched_policy(pthread_attr_t *attr, int *policy)`
 2. `pthread_attr_setsched_policy(pthread_attr_t *attr, int policy)`



Operating System Examples

Unix, Linux, Windows, and Solaris scheduling

Operating System Examples

- Linux scheduling
- Windows scheduling
- Solaris scheduling

BSD Unix Scheduling

- BSD Unix uses multilevel feed back queue approach, with 32 run queues
- System processes use run queues **0** through **7**
- User processes use run queues 8 through 31
- The dispatcher selects a process from higher priority queue
- Within a queue RR scheduling is used
- Time quantum vary among implementation, but all are less than 100ms
- Each process has an external nice priority
- The nice priority can vary between -20 and 20, -20 is the highest, 20 is the lowest user priority
- Each process's current priority id recalculated after each time quantum

Linux Scheduling Through Version 2.5

- Prior to kernel version 2.5, ran variation of standard UNIX scheduling algorithm
- Version 2.5 moved to constant order $O(1)$ scheduling time
 - Preemptive, priority based
 - Two priority ranges: time-sharing and real-time
 - Real-time range from 0 to 99 and nice value from 100 to 140
 - Map into global priority with numerically lower values indicating higher priority
 - Higher priority gets larger q
 - Task run-able as long as time left in time slice (**active**)
 - If no time left (**expired**), not run-able until all other tasks use their slices
 - All run-able tasks tracked in per-CPU **runqueue** data structure
 - Two priority arrays (active, expired)
 - Tasks indexed by priority
 - When no more active, arrays are exchanged
 - Worked well, but poor response times for interactive processes

Linux Scheduling in Version 2.6.23 +

Completely Fair Scheduler (CFS)

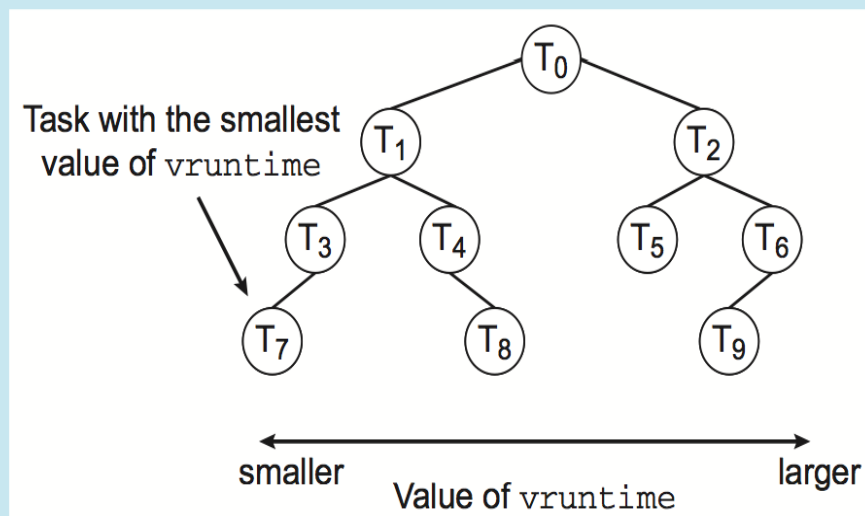
- Scheduling classes
 - Each has specific priority
 - Scheduler picks highest priority task in highest scheduling class
 - Rather than quantum based on fixed time allotments, based on proportion of CPU time
 - Two scheduling classes included, others can be added
 - default
 - real-time

Linux Scheduling in Version 2.6.23 +

- Quantum calculated based on **nice value** from -20 to +19
 - Lower value is higher priority
 - Calculates **target latency** – interval of time during which task should run at least once
 - Target latency can increase if say number of active tasks increases
- CFS scheduler maintains per task **virtual run time** in variable **vruntime**
 - Associated with decay factor based on priority of task – lower priority is higher decay rate
 - Normal default priority yields virtual run time = actual run time
- To decide next task to run, scheduler picks task with lowest virtual run time

CFS Performance

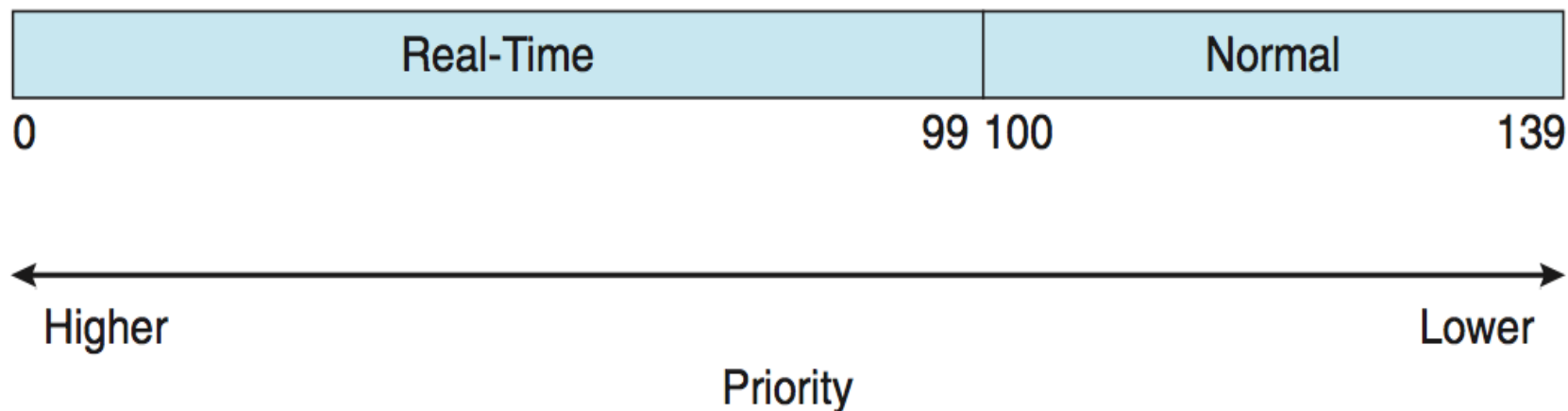
The Linux CFS scheduler provides an efficient algorithm for selecting which task to run next. Each runnable task is placed in a red-black tree—a balanced binary search tree whose key is based on the value of `vruntime`. This tree is shown below:



When a task becomes runnable, it is added to the tree. If a task on the tree is not runnable (for example, if it is blocked while waiting for I/O), it is removed. Generally speaking, tasks that have been given less processing time (smaller values of `vruntime`) are toward the left side of the tree, and tasks that have been given more processing time are on the right side. According to the properties of a binary search tree, the leftmost node has the smallest key value, which for the sake of the CFS scheduler means that it is the task with the highest priority. Because the red-black tree is balanced, navigating it to discover the leftmost node will require $O(\lg N)$ operations (where N is the number of nodes in the tree). However, for efficiency reasons, the Linux scheduler caches this value in the variable `rb_leftmost`, and thus determining which task to run next requires only retrieving the cached value.

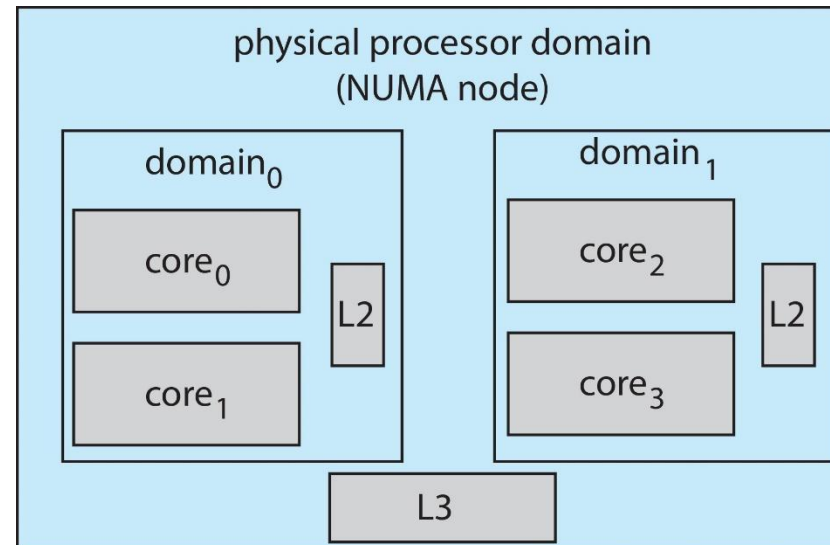
Linux Scheduling

- Real-time scheduling according to POSIX.1b
 - Real-time tasks have static priorities
- Real-time plus normal map into global priority scheme
- Nice value of -20 maps to global priority 100
- Nice value of +19 maps to priority 139



Linux Scheduling

- Linux supports load balancing, but is also NUMA-aware
- **Scheduling domain** is a set of CPU cores that can be balanced against one another.
- Domains are organized by what they share (i.e., cache memory.) Goal is to keep threads from migrating between domains



Windows Scheduling

- Windows use a priority based, preemptive scheduling algorithm
- Windows scheduler ensures that highest priority thread will always run
 - Thread runs until
 1. blocks
 2. uses time slice
 3. preempted by higher-priority thread
- Windows dispatcher uses a 32-level priority scheme
- Priorities are divided into two classes
 - Variable class (priorities from 0 to 15)
 - Real time class (priorities from 16 to 31)
- Priority 0 is memory-management thread
- Dispatcher uses a queue for each scheduling priority
- If no run-able thread, runs **idle thread**

Windows Scheduling: Priority Classes

- Win 32 API identifies several priority classes. These are;
 - REALTIME_PRIORITY_CLASS
 - HIGH_PRIORITY_CLASS
 - ABOVE_NORMAL_PRIORITY_CLASS
 - NORMAL_PRIORITY_CLASS
 - BELOW_NORMAL_PRIORITY_CLASS
 - IDLE_PRIORITY_CLASS
- Within each of these priority there is a relative priority which are;
 - TIME_CRITICAL, HIGHEST, ABOVE_NORMAL, NORMAL, BELOW_NORMAL, LOWEST, IDLE
- Priority class and relative priority combine to give numeric priority
- Base priority is NORMAL within the class

Windows Scheduling: Priority Classes

- If quantum expires, priority lowered, but never below base
- If wait occurs, priority boosted depending on what was waited for
- Foreground window given 3x priority boost
- Windows 7 added **user-mode scheduling (UMS)**
 - Applications create and manage threads independent of kernel
 - For large number of threads, much more efficient
 - UMS schedulers come from programming language libraries like C++ **Concurrent Runtime** (ConcRT) framework

Windows Priorities

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1

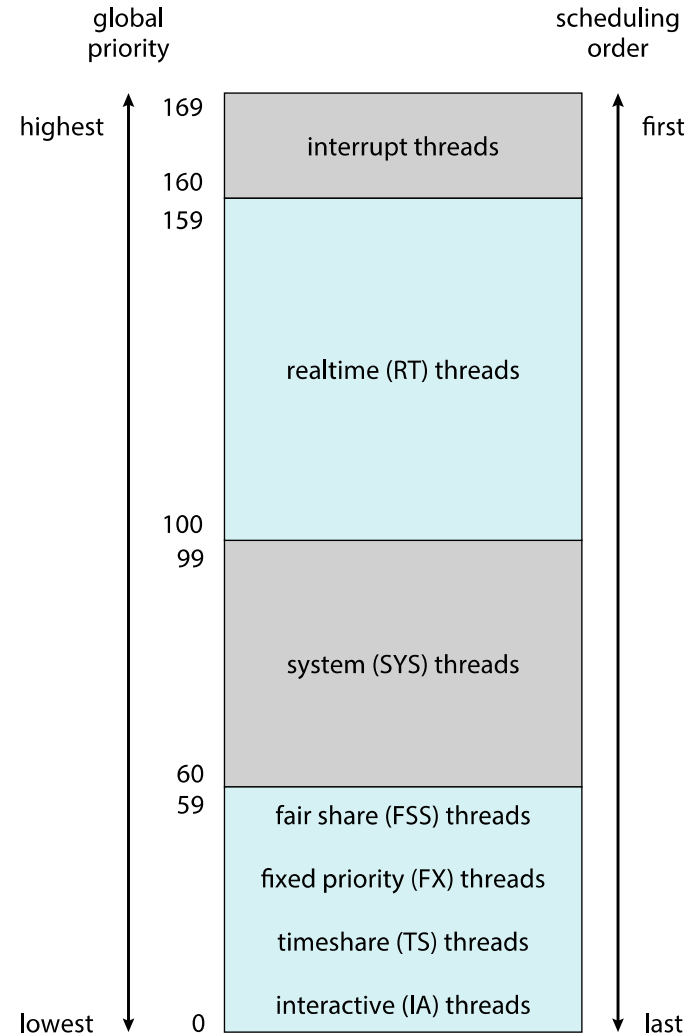
Solaris Scheduling

- Solaris uses priority-based process scheduling
- It has six classes of priority, which are;
 - Real time (RT)
 - System (SYS)
 - Fair Share (FSS)
 - Fixed priority (FP)
 - Time sharing (TS) (default)
 - Interactive (IA)
- Given thread can be in one class at a time
- Each class includes different set of priorities
- Each class has its own scheduling algorithm
- Real time processes has highest priority and interactive and time-sharing processes have least priorities
- Solaris uses system class to run kernel processes, such as scheduler
 - System class is reserved for kernel processes

Solaris Dispatch Table

priority	time quantum	time quantum expired	return from sleep
0	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59	20	49	59

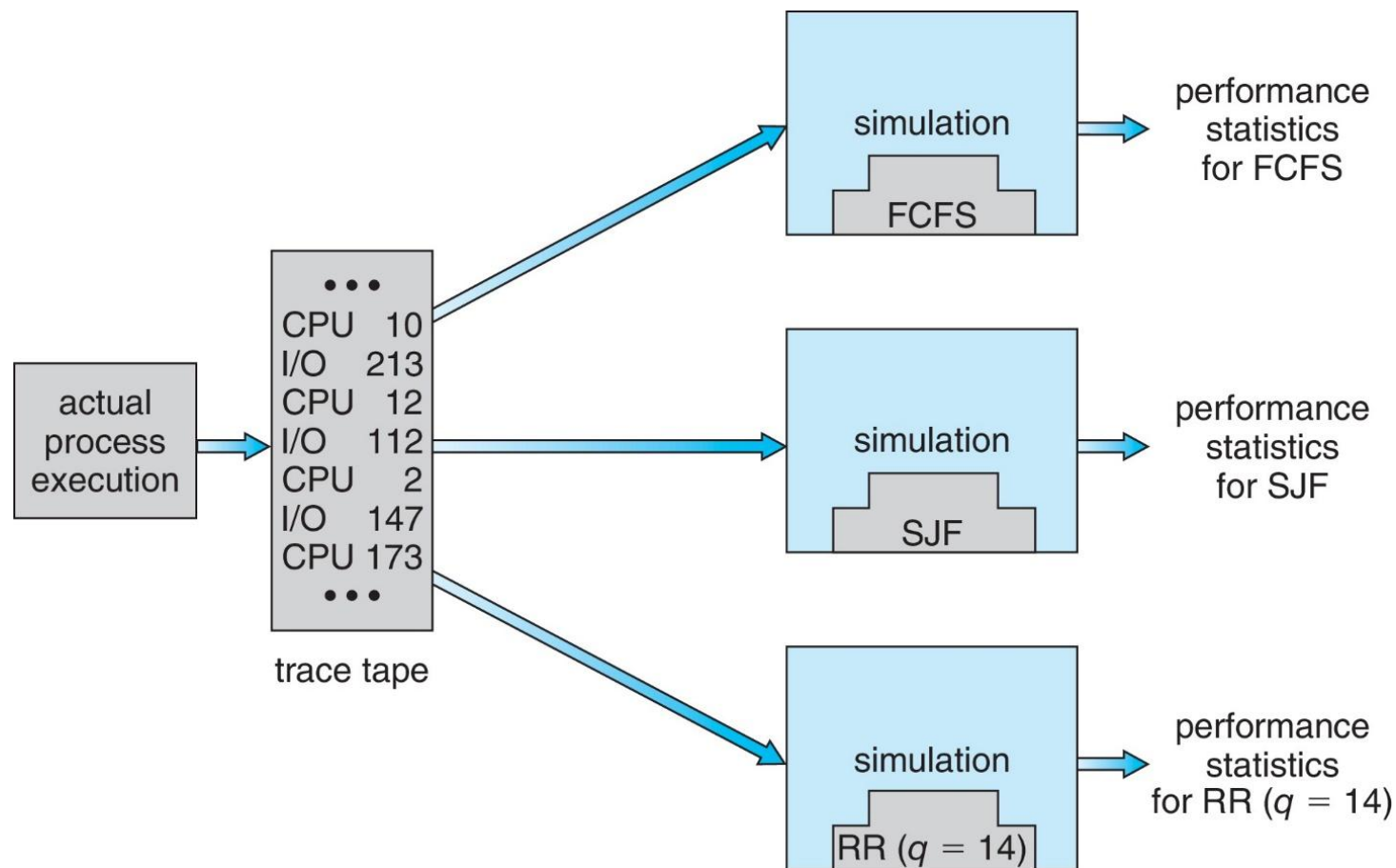
Solaris Scheduling



Solaris Scheduling

- Scheduler converts class-specific priorities into a per-thread global priority
 - Thread with highest priority runs next
 - Runs until (1) blocks, (2) uses time slice, (3) preempted by higher-priority thread
 - Multiple threads at same priority selected via RR

Evaluation of CPU Schedulers by Simulation



Implementation

- Even simulations have limited accuracy
- Just implement new scheduler and test in real systems
 - High cost, high risk
 - Environments vary
- Most flexible schedulers can be modified per-site or per-system
- Or APIs to modify priorities
- But again environments vary